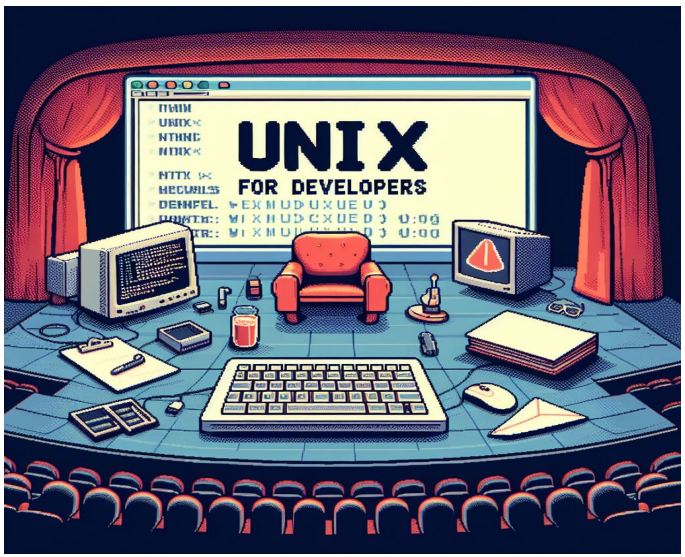
Shell Scripting & Programming

Benedict Reuschling

November 10, 2025



Shell Programming

This chapter explains the creation of scripts for the Unix shell. Familiar programming concepts like variables, control structures, and branches are applicable in shell scripts as well. The presentation covers both basic concepts and shell-specific constructs.

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using `if`
   - Multiple Choice Using `case`
   - Loops: The `for` Loop
   - Loops: The `while` Loop
4. Debugging Shell Scripts

Overview

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using `if`
   - Multiple Choice Using `case`
   - Loops: The `for` Loop
   - Loops: The `while` Loop
4. Debugging Shell Scripts

Basics

As shown in the chapter about shells, usage is either interactively via the console, but also reads instructions from a file line by line.

```
$ cat date_and_current_dir.txt
date
ls
$
$ sh < date_and_current_dir.txt
Fri Nov  2 15:14:27 CET 2012
123.x     abc1      xy        xy2   xy2345.c date_and_current_dir.txt
$
```

The direct passing of a file as an argument to the shell is also possible.

Basics - Creating Shell Scripts

Direct execution is even simpler when the file is executable.

```
$ chmod +x date_and_current_dir.txt
$ date_and_current_dir.txt
Fri Nov  2 15:14:27 CET 2012
123.x     abc1      xy        xy2   xy2345.c date_and_current_dir.txt
$
```

For shell scripts, the file extension `.sh` has become standard. In Unix, a shell script does not need to have this extension, though. Picking a good filename helps identifying it from other scripts.
How to execute this script from any directory without copying that file there first?

Basics - Making Shell Scripts Globally Available

The /bin directory contains crucial Unix commands and is accessible system-wide. Ordinary users lack write access to this directory for security reasons. Instead, they can create their own directory in their home directory (/home/bin). Any scripts in that directory become available system-wide without requiring the full path. To enable this functionality, ensure that the $PATH variable includes the directory $HOME/bin.

```
$ mkdir $HOME/bin
$ mv date_and_current_dir.txt $HOME/bin/dacd.sh
$ PATH=$PATH:$HOME/bin
$ cd /tmp
$ dacd.sh
...
$
```

The variable HOME indicates the home directory of the logged-in user. Meanwhile, the variable PATH establishes the search path for executable files.

Basics - Naming Conventions for Shell Scripts

Warning:

Because shell programs can take any name the user wants, they can also called like Unix system programs. Avoid this for the following reasons:

- shell programs shadow system programs; the latter are not available anymore in this case
- endless loops can occur

The following examples illustrate this.

Example: Shadowing of a System Program via Shell Script

```
$ mv dacd.sh ps
$ PATH=.:$PATH
$ ps
Fri Nov  2 17:38:27 CET 2012
123.x      abc1       xy         xy2    xy2345.c
$
```

The Unix command ps can no longer run as usual. To get the original ps command back, specify the full path to the executable.

```
$ /bin/ps
PID TT  STAT      TIME COMMAND
41718  0  Rs    0:00.09 -zsh (zsh)
41825  0  R+    0:00.00 ps
```

Example: Endless Loop

```
$ mv dacd.sh ls
$ cat ls
date
ls
$ ls
Fri Nov  2 16:17:46 CET 2012
Fri Nov  2 16:17:47 CET 2012
Fri Nov  2 16:17:48 CET 2012
Fri Nov  2 16:17:49 CET 2012
Fri Nov  2 16:17:50 CET 2012
...
^C
$
```

Prevent this issue by **consistently using the complete path to system programs in shell scripts** (e.g., `/bin/<command>`). When delving into shell scripting, adopt this practice from the beginning to build familiarity. The `which` command aids in identifying the path to system programs (unless already overridden).

Basics - Define the Interpreter of the Shell Script

Different shells have functions to support the user to write them. The interpreter indicates for which shell this script works. The first line of the shell indicates this. This does **not** represent a comment.

```
$ cat helloworld.sh
#!/bin/sh
echo Hello World!
$
```

This script should run `sh` in the `/bin` directory, which serves as the default interpreter for shell scripts. While other shells (e.g., `#!/bin/bash`) may offer enhanced functionality, such as regular expressions, these scripts are less portable. On a different Unix system, the preferred shell (e.g., bash and others) may not be accessible or located in a different path (e.g., `/usr/local/bin/bash` as seen in the BSDs). In such cases, the script may not run without modifications. In contrast, the recommended interpreter is `/bin/sh` **ensuring availability on all Unix systems**.

Basics - Comments

Ignored lines by the interpreter are comments. In shell scripts, placea # at the beginning of the line. The interpreter ignores the rest of the line (no execution).

```
$ cat helloworld.sh
#!/bin/sh
# This is a comment
echo Hello World!
$
```

Overview

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using `if`
   - Multiple Choice Using `case`
   - Loops: The `for` Loop
   - Loops: The `while` Loop
4. Debugging Shell Scripts

Variables

Values and data, akin to files, are obtained from variables. Shell scripts lack explicit data types such as int or string. Despite this absence, comparisons and calculations are possible, and users can check results as in typed programming languages. Access variables using their names. Retrieve a variable's value (if any) by prepending the name with a $ (dollar sign) and enclosing the name in curly braces { and }.

Types of Variables in Shell Scripts

Basically, three types of variables are available:

- positional parameters
- special parameters
- user defined variables

The following slides explain them.

Types of Variables: Positional Parameters

A positional parameter is a variable in a shell script, with their values specified at the command line, represented by $1, $2, $3, ... $9. Possible are 9 such parameters.

```
$ cat positionparameter.sh
#!/bin/sh
echo 1st parameter: $1
echo 2nd parameter: $2
echo 3rd parameter: $3
$ positionparameter.sh first second number3
1st parameter: first
2nd parameter: second
3rd parameter: number3
$
```

A special case is the variable $0. It represents the name and full path of the script itself, in our example `positionparameter.sh`.

Using More Than 9 Positional Parameters

Often, the number of supplied arguments to a shell script exceeds the predefined 9 positional parameters. In this case, the `shift` command can help. The `shift` command moves the current value in the positional parameter to the left (shifting).

Example:

```
$ cat shifter.sh
#!/bin/sh
echo Parameters before: \$1=$1, \$2=$2, \$3=$3
shift
echo Parameters after: \$1=$1, \$2=$2, \$3=$3
$ ./shifter.sh one two three
Parameters before: $1=one, $2=two, $3=three
Parameters after: $1=two, $2=three, $3=
```

`shift` can also move more places at once by supplying the number as a parameter to it.

More than 9 Positional Parameters

`shift` can move parameters more places to the left when provided a number greater than one.

Example:

```
$ cat shifter2.sh
#!/bin/sh
echo Parameters before: \$1=$1, \$2=$2, \$3=$3
shift 9
echo Parameters after: \$1=$1, \$2=$2, \$3=$3
$ ./shifter2.sh A B C D E F G H I J K L
Parameters before: $1=A, $2=B, $3=C
Parameters after: $1=J, $2=K, $3=L
```

Types of Variables: Special Parameters

The following special parameters are available:

- `$#` the number of arguments supplied to the script
- `$*` the full string of provided arguments
- `$$` the process ID of the shell script

```
$ cat special.sh
#!/bin/sh
echo Number of arguments: $#
echo The arguments are: $*
$ ./special.sh one two three End
Number of arguments: 4
The arguments are: one two three End
$
$ cat scriptPID.sh
#!/bin/sh
echo $$
$ scriptPID.sh
33742
$
```

A robust shell script often uses these variables (more about that later).

Types of Variables: User Defined Variables

Users create variables using the form

```
name=value
```

Allowed are even empty values.

```
$ echo $FIRSTNAME

$ FIRSTNAME=Susan
$ echo $FIRSTNAME
Susan
$ AGE=23
$ echo $AGE
23
```

Important: There must be *no* spaces on the left and right of the = sign. Variable names must begin with a letter or an _ (underscore), followed by combinations of letters, numbers, and underscores. Command names are not allowed for variable names.

Assigning Values to Variables

There are possibilities to fill variables with values:

- User input via keyboard using the `read` command
- Output redirection of commands into the variable
- Assigning values of other variables

See the different types in more detail on the following slides.

User Input from Keyboard via `read`

Using the `read` command, a prompt appears at which the user can make inputs. The syntax is:

```
read nameofvariable
```

The following example asks the user for their name and echoes it to the shell afterwards.

```
$ cat readname.sh
#!/bin/sh
echo What is your name?
read name
echo Hello $name
$ readname.sh
What is your name?
Benedict
Hello Benedict
```

Asking the user for information makes programs more interactive.

Special case: Passwords

When asking the user for passwords and other sensitive information the input should not be visible to anyone snooping over their shoulder. To prevent this, change the terminal via the `stty(1)` command before the input so no characters appear on the screen. After finishing the input of the password another call to `stty(1)` resets the terminal back to normal.

```
$ cat readhidden.sh
#!/bin/sh
echo username:
read user
echo password:
stty -echo
read password
stty echo
$
$ readhidden.sh
username:
bcr
password:

$
```

Output Redirection of Commands into Variables

Assign the output of commands to a variable by enclosing them in backticks. The syntax is as follows:

```
variable=`command`
```

The following example demonstrates this:

```
$ cat timeofday.sh
timeofday=`date|cut -c16-23`
echo $timeofday
$ timeofday.sh
12:37:18
```

Directly including output from the echo command without assigning to a variable makes the following script even shorter:

```
$ cat timeofday2.sh
echo `date|cut -c16-23`
$ timeofday2.sh
12:37:25
```

Equivalency

The following statements are equal:

`variable=`command`` and `variable=$(command)`

The `variable=$(command)` is more common in shell scripts nowadays as it allows nesting of such statements. We'll use the form `variable=$(command)` from now on.

Calculate Values with `expr`

Mathematical calculations are frequently found in shell scripts. As another example of output redirection into variables, we will look at the `expr` program. It allows basic integer (non-fractional) arithmetic calculations.
The syntax of `expr` is this:

```
expr operand1 operator operand2
```

Separate operand and operator with a space.
The following operators are available:

- `+` Addition
- `-` Subtraction
- `\*` Multiplication (quoted otherwise used as wildcard)
- `/` Division
- `%` Modulo

Example: Calculate Values with expr

The example that follows demonstrates the usage of expr:

```
$ cat minicalc.sh
echo First number:
read number1
echo Second number:
read number2
echo "Result of adding the two: $(expr $number1 + $number2)"
echo "Result of subtracting the two: $(expr $number1 - $number2)"
echo "Result of dividing the two: $(expr $number1 / $number2)"
echo "Result of multiplying the two: $(expr $number1 \* $number2)"
$ minicalc.sh
First number:
8
Second number:
4
Result of adding the two: 12
Result of subtracting the two: 4
Result of dividing the two: 2
Result of multiplying the two: 32
```

Note: Division by zero is not checked here, but revisited in the section about control structures.

Another Syntax for Calculations using Integers

The following syntax is also allowed by POSIX[1] in `/bin/sh` and other shells:

```
#/bin/sh
mycounter=0
echo $mycounter
mycounter=$((mycounter+3))
echo $mycounter
```

The output is:

0
3

The same works for -, *, /, and %.
Errors emitted when doing non-integer arithmetic are:

```
lines=$((lines/2.2))
sh: lines/2.2: syntax error: invalid arithmetic operator (error token is "
```

[1] https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html

Defining Default Values

Use default values to assign a variable a specific value in cases where the variable would otherwise be unset. For instance, a script might take a parameter ($1) and assign it to an internal variable for processing. What if the parameter is absent? The script becomes more robust by defining a default value instead. In situations where the variable is empty, the default value takes its place. The syntax for this is as follows:

```
othervar=${variable-default_value}
```

If `$variable` is unset, the value `default_value` gets assigned to `othervar`. The default value can be a string enclosed in single or double quotes. Importantly, the default value gets disregarded when `$variable` is not empty. This approach works with all variables, including positional parameters.

Assigning Values from Other Variables

Assigning values from other variables is simple and used like other programming languages. Even chaining variables to a new one is possible.

```
$ cat readfullname.sh
#!/bin/sh
echo What is your first name?
read firstname
echo What is your last name?
read lastname
fullname="${firstname} ${lastname}"
echo Your full name is ${fullname}
$
$ readfullname.sh
What is your first name?
Susi
What is your last name?
Sunshine
Your full name is Susi Sunshine
```

In this case, put `firstname` and `lastname` into quotes, so that the space is also part of the new variable `$fullname`.

Overview

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using `if`
   - Multiple Choice Using `case`
   - Loops: The `for` Loop
   - Loops: The `while` Loop
4. Debugging Shell Scripts

Conditional Execution using if then elif else

Control the program flow by using if statements.
The syntax is:

```
if condition1
then
    condition is zero, true, 0
    execute all commands until the elif or else expression
elif condition2
then
     execute all commands until the else expression
else
    execute command(s) in case the statement(s) before were no
    run all commands until fi
fi
```

The elif and else part is optional. Further conditions placed within the if, elif and/or else branches and allow nested conditionals.

Conditional Execution - Formulating Conditions with `test`

The `test`[2] program checks whether the supplied parameters are `true` and returns zero (0) when they do, otherwise false or nonzero. Another form is to enclose conditionals by using `[` and `]`.

Syntax:

```
test expression
```

or

```
[ Expression ]
```

`test(1)` describes more conditions that `test` can check.

[2] Due to that, never name any shell script `test`.

Valid Conditions Which `test` Checks

`-f File`: Checks whether `File` exists, regardless of the type

`-n String`: Returns `true`, when the length of the string is nonzero

`str1 = str2`: Checks whether `str1` and `str2` (strings) are identical

`str1 != str2`: Checks whether `str1` is not equal to `str2`

`n1 -eq n2`: Checks whether `n1` is equal to `n2` (number) (**eq**ual)

`n1 -ne n2`: Checks whether `n1` is not equal to `n2` (**n**ot **e**qual)

`n1 -gt n2`: Checks whether `n1` is greater than `n2` (**g**rea**t**er)

`n1 -ge n2`: Checks whether `n1` is greater or equal to `n2` (**g**reater or **e**qual)

`n1 -lt n2`: Checks whether `n1` is less than `n2` (**l**ess **t**han)

`n1 -le n2`: Checks whether `n1` is less than or equal to `n2` (**l**ess or **e**qual)

`Expression1 -a Expression2`: Returns `true` when `Expression1` *and* Expression2 are true

`Expression1 -o Expression2`: Returns `true` when `Expression1` *or* Expression2 (or both) are true

Example for Conditional Execution

```
$ cat division.sh
#!/bin/sh
echo Enter dividend:
read dividend
echo Enter divisor:
read divisor
if [ $divisor -ne 0 ]; then
   echo "The result of the division is $(expr $dividend / $divisor)."
else
   echo "Division by zero is not allowed!"
   exit 1;
fi
```

Example Output

```
$ division.sh
Enter Dividend:
12
Enter Divisor:
4
The result of the division is 3.
$ division.sh
Enter Dividend:
12
Enter Divisor:
0
Division by zero is not allowed!
```

Overview

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using `if`
   - Multiple Choice Using `case`
   - Loops: The `for` Loop
   - Loops: The `while` Loop
4. Debugging Shell Scripts

Multiple Choice Using case

Write long `if` statements with a lot of selections using the `case` statement.

Syntax:

```
case $Variable in
      Pattern1) command
                ...
                last command;;
      Pattern2) command
                ...
                last command;;
      PatternN) command
                ...
                last command;;
             *) command
                ...
                last command;;
 esac
```

Example for case

```
$ cat lunch.sh
#!/bin/sh
echo "Select whether you want 1) meat or 2) vegetarian lunch."
read choice
case $choice in
    1) echo "No meat available today.";;
    2) echo "No vegetarian menu available today.";;
    *) echo "Not a valid choice!"
esac
$ lunch.sh
Select whether you want 1) meat or 2) vegetarian lunch.
1
No meat available today.
$ lunch.sh
Select whether you want 1) meat or 2) vegetarian lunch.
2
No vegetarian menu available today.
$ lunch.sh
Select whether you want 1) meat or 2) vegetarian lunch.
0
Not a valid choice!
```

More valid patterns besides * are ? and [and/or].

Checking for Substrings with case

A common problem we need to solve is whether a string contains a certain substring. For example, whether *Dominion* contains *minion* or *ini*. The following case construct detects the substring:

```
#!/bin/sh
WORD="Administrator"

case "$WORD" in
  *ministrat*) echo "$WORD contains 'ministrat'" ;;
  *ini*) echo "$WORD contains 'ini'" ;;
  *) echo "No matching substring found" ;;
esac
```

To check for a substring at the beginning of a word, omit the leading * from the pattern. For example, to check whether a string is a student account of h_da (begins with *st*), you can use:

```
st*)
```

That way, the word *Administrator* will not cause a pattern match.

Overview

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using if
   - Multiple Choice Using case
   - Loops: The for Loop
   - Loops: The while Loop
4. Debugging Shell Scripts

Loops: the `for` Loop

The `for` loop creates a variable and assigns it the values listed after it one by one. For every assignment (iteration) one or more commands run. This will repeat until all elements of the list have run out.
Syntax:

```
for { nameofvariable } in { list }
do
   command1 (for every element of the list)
   commandN
done
```

Not strictly necessary to use or change the current list element in the loop body. The `list` can contain numbers, words and even filenames.

for Loop Example

This example extracts files in parallel by starting each extraction as a background process (job).

```
$ cat extract.sh
#!/bin/sh
for file in lecture1.tar.gz lecture2.tar.gz lecture.tar.gz
do
   tar -xvf $file &
done
```

Removing the & extracts all listed files one after the other.

Loops: The for Loop

The following syntax is closer to the one known from C/C++:

```
for (( Expression1; Expression2; Expression3 ))
do
        repeat command(s) between do and done
        until Expression2 is to true
done
```

Allowed by bash (and other shells), but not in `#!/bin/sh`. When trying this with sh produces the following error: `Syntax error: Bad for loop variable`

To solve this problem, `seq` can create a series of numbers and assign them to the loop variable (here: `i`) one by one:

```
#!/bin/sh
for i in $(seq 0 4)
do
    echo $i
done
```

Example for Loop in sh and bash

```
$ cat multiinsh.sh
#!/bin/sh
for i in $(seq 1 3)
do
    echo "$i * $i = $(expr $i \* $i)"
done
$ sh multiinsh.sh
1 * 1 = 1
2 * 2 = 4
3 * 3 = 9
```

```
$ cat multiinbash.sh
#!/bin/bash
for (( i = 1 ; i <= 3; i++ ))
do
    echo "$i * $i = $(expr $i \* $i)"
done
$ multiinbash.sh
1 * 1 = 1
2 * 2 = 4
3 * 3 = 9
```

Overview

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using `if`
   - Multiple Choice Using `case`
   - Loops: The `for` Loop
   - Loops: The `while` Loop
4. Debugging Shell Scripts

Loops: The while Loop

The condition of the `while` loop runs until the defined condition(s) is `true`.

Syntax:

```
while [ Condition(s) ]
do
        Command1
        Command2
        CommandN
done
```

Example:

```
$ cat countdown.sh
#!/bin/sh
countdown=10
while [ $countdown -ne 0 ]
do
    echo "$countdown"
    countdown=$(expr $countdown - 1)
done
```

Combined Example from Practical Experience

```
$ cat restart.sh
#!/bin/sh
loop=1
while [ $loop -eq 1 ]; do
    echo -n "Do you want to restart the SSH service now (y/n)? "
    read choice
    case "$choice" in
        [yY]|yes|Yes)
            ssh_binary=$(which sshd)
            $ssh_binary -t
            checkresult=$?
            if [ $checkresult -eq 0 ]; then
                sudo service ssh restart
            fi
            loop=0
            ;;
        [nN]|no|No) loop=0 ;;
        *)
            echo "Wrong answer given, please repeat";
            loop=1 ;;
    esac
done
```

Overview

1. Basics
2. Variables
   - Positional Parameters
   - Special Parameters
   - User Defined Variables
3. Control Structures
   - Conditional Execution using `if`
   - Multiple Choice Using `case`
   - Loops: The `for` Loop
   - Loops: The `while` Loop
4. Debugging Shell Scripts

Debugging Shell Scripts

"Even now I don't know how you debug shell scripts."
– Stephen Bourne, inventor of sh, June 2015 BSDCan Keynote,
URL: `https://www.youtube.com/watch?v=2kEJoWfobpA`

Debugging Shell Scripts

To debug shell scripts there are some possibilities supplied by the shell. During the debugging phase, put these at the end of the interpreter line:

- `-e` strict mode, aborts the script execution on all **e**rrors (exit code nonzero)
- `-u` Detect **u**nset variable usage
- `-v:` logs raw input, including unexpanded variables and comments (**v**erbose)
- `-x:` After each command is (e**x**ecute) /(expanded) the shell shows the completed expression

Instead of the interpreter line, provide these parameters together or separately to the shell during execution:

```
$ sh -x demo.sh
$ sh -v demo.sh
$ sh -xv demo.sh
```

Usage Example for -x

This example tests the correct evaluation of the `expr` clause.

```
$ cat countdown.sh
#!/bin/sh -x
countdown=2
while [ $countdown -ne 0 ]
do
    echo "$countdown"
    countdown=$(expr $countdown - 1)
done
$ countdown.sh
+ countdown=2
+ '[' 2 -ne 0 ']'
+ echo 2
2
++ expr 2 - 1
+ countdown=1
+ '[' 1 -ne 0 ']'
+ echo 1
1
++ expr 1 - 1
+ countdown=0
+ '[' 0 -ne 0 ']'
```

Usage Example for -v

These examples check the proper program flow.

```
$ cat countdown.sh
#!/bin/sh -v
countdown=2
while [ $countdown -ne 0 ]
do
    echo "$countdown"
    countdown=$(expr $countdown - 1)
done
$ countdown.sh
#!/bin/sh -v
countdown=2
while [ $countdown -ne 0 ]
do
    echo "$countdown"
    countdown=$(expr $countdown - 1)
done
2
expr $countdown - 1
1
expr $countdown - 1
```

Debugging help with `set`

Use `set -e` in shell scripts to ensure the right program flow and exit on any errors.
The `set` built-in at the beginning of the script provides some helpful debugging output for specific problems. The options to `set` provide the actual functionality:

`set -o pipefail` Abort when a command if a pipe fails

`set -o nounset` Abort when accessing an unset variable

`set -o errexit` Exit when a command in the script fails

`set -o xtrace` Run with `TRACE=1 ./myscript.sh` for debug output, see [SHBP]

Without try-catch mechanisms available in the shell, recovering from failures is difficult. Abort script execution in those cases.
An online linter is available at `https://www.shellcheck.net` [SHCHK].

Further Information



[LSST] Vivek G. Gite
Linux Shell Scripting Tutorial - A Beginner's handbook
http://freeos.com/guides/lsst/index.html



[RHWSH] Jürgen Wolf
Shell-Programmierung - Einführung, Praxis, Referenz
http://openbook.rheinwerk-verlag.de/shell_programmierung



[FSHUNIT] Shell Unit Testing Framework
shUnit 2
https://github.com/kward/shunit2



[SHBP] The Sharat's Blog: Shell Script Best Practices
https://sharats.me/posts/shell-script-best-practices/



[SHCHK] Shell Check Online Linter
https://www.shellcheck.net